2.11A C语言不检测数据溢出的特性

定位：计算机组成原理 · 第 2 章 数据表示（2.11A） | 答疑卡 | 配合 2.11 C语言中的数据类型及其转换 与 3.5 定点数的加减运算——溢出检测 阅读

一句话核心：C 语言把溢出检测交给程序员；无符号溢出是模回绕，有符号溢出是未定义行为，数组越界完全不检查

一、整数溢出

有符号整数

- C 标准规定为未定义行为（UB）
- 可能回绕、崩溃、被优化掉，行为不确定
- 不能依赖“溢出后变成某个值”

无符号整数

- 明确定义为模 2^N 回绕
- 例：`unsigned int` 最大值加 1 变成 0

```
int a = INT_MAX;
a++;  // 有符号溢出，UB

unsigned int b = UINT_MAX;
b++;  // 无符号回绕，结果为 0
```

408 口径

- 无符号溢出看 CF
- 有符号溢出看 OF
- C 语言的 UB 不是 408 的考试口径；做题按补码运算规则

二、数组越界

- C 不检查数组下标
- `arr[100]` 在数组只有 10 个元素时也可能编译、运行
- 越界可能破坏栈/堆，是安全漏洞的常见来源
- 408 不考数组越界的具体攻击，只考“C 不做运行时检查”这个结论

三、为什么这样设计

- C 诞生于 1970 年代，强调“信任程序员”和零成本抽象
- 溢出检测需要额外指令和分支，C 选择不做
- 需要检查时，由程序员手动判断或使用工具

四、C 程序员的自保手段

1. 运算前判断边界，如 `if (a > INT_MAX - b)`
2. 使用编译器检查：`-ftrapv`、`-fsanitize=undefined`、`-fsanitize=address`
3. 使用安全库/函数：C23 的 `ckd_add` 等
4. 安全性优先时，选择带溢出检查的语言或类型

一句话总结：C 语言把溢出的检测责任交给程序员和工具链，编译器和运行时不会自动兜底

知识关联：C 类型转换（2.11 C语言中的数据类型及其转换）；补码溢出（3.5 定点数的加减运算——溢出检测）；CF/OF（5.8 指令的操作类型）